

Optimización de Algoritmos Depth-First Search en Matrices 2D mediante un Procesador RISC-V con Aceleramiento por Hardware

Gabriel Abarca Aguilar, Jesús Alberto Castro Murillo, José Fabio Jaramillo Cordero,
Moisés Leiva Solano y Noel Antonio Pérez Cáceres
Escuela de Ingeniería en Electrónica
Instituto Tecnológico de Costa Rica
Costa Rica

Resumen—*Depth-First Search* (DFS) is a fundamental traversal primitive over 2D matrices, underlying problems such as connected-component labeling, *flood fill*, and pattern search on grids. Its sequential nature and irregular memory accesses limit performance on general-purpose processors. This work proposes to optimize DFS algorithms over 2D matrices using a RISC-V processor with hardware acceleration, modeled at a high level in SystemC to enable early architectural exploration. Based on the analysis of several DFS problems, the dominant computation pattern is identified as a candidate for specialization, and an experimental methodology is outlined to contrast the solution against a baseline RISC-V execution in terms of throughput, latency, and resource usage.

Index Terms—Depth-First Search, RISC-V, hardware acceleration, 2D matrices, SystemC, architectural exploration.

I. INTRODUCCIÓN

Depth-First Search (DFS) es uno de los algoritmos de recorrido más fundamentales en computación, y es el núcleo de múltiples problemas definidos sobre matrices 2D [1]. Al interpretar cada celda como un nodo conectado a sus vecinos, DFS permite descubrir conectividad, regiones y trayectorias dentro de la cuadrícula. Esta representación aparece en dominios distintos: *pathfinding* sobre mapas de celdas en robótica y videojuegos, etiquetado de componentes conexas y *flood fill* en procesamiento de imágenes, y planificación en espacios discretos.

La eficiencia de DFS depende de cómo se plantee el problema sobre la matriz. En un recorrido simple, donde cada celda se visita una sola vez, el costo es lineal: $O(m \cdot n)$ en tiempo y espacio para una matriz de $m \times n$ [1]. Sin embargo, muchos problemas lanzan una búsqueda independiente desde cada celda, explorando todos los caminos posibles hasta cierta profundidad. En estos casos el costo deja de ser lineal: con $m \cdot n$ orígenes, un factor de ramificación b y caminos de longitud máxima L , el trabajo escala como $O(m \cdot n \cdot b^L)$, de forma exponencial respecto a la longitud del camino. Este crecimiento convierte al recorrido DFS en el cuello de botella de estas aplicaciones y motiva explotar el paralelismo entre las búsquedas independientes.

Estos costos se agravan sobre hardware de propósito general. El recorrido de matrices y grafos presenta accesos a

memoria irregulares y poca intensidad computacional, lo que degrada el desempeño en CPUs, cuyas jerarquías de caché no logran explotar localidad [2], [3]. Además, en plataformas con memoria reducida la pila y el registro de visitados no siempre caben en la memoria interna, por lo que los datos se transfieren de forma repetida y el tiempo de transferencia domina sobre el de cómputo [2]. La naturaleza secuencial de DFS también limita el paralelismo: al compartir el estado del recorrido, múltiples hilos sobre una misma memoria aumentan el *overhead* por contención en lugar de mejorar el rendimiento [2]. Por ello se han adoptado arquitecturas especializadas que adaptan el *datapath* y la memoria al patrón de acceso del recorrido [3].

Este trabajo aborda la optimización de algoritmos DFS sobre matrices 2D mediante un procesador RISC-V con aceleramiento por hardware, modelado en SystemC. La idea central es identificar, a partir del código de varios problemas DFS, el patrón de cómputo dominante —expansión de vecinos, verificación de visitado y actualización de la pila— y especializar el procesador para ejecutarlo de forma eficiente, dejando al núcleo RISC-V como secuenciador. Las contribuciones son tres: (i) un análisis del patrón de cómputo común a varios problemas DFS y la identificación de candidatos a aceleración; (ii) una propuesta de arquitectura y su modelo en SystemC, que habilita la exploración del espacio de diseño antes del RTL; y (iii) una metodología experimental para contrastar la solución frente a una ejecución RISC-V base, con métricas de *throughput*, latencia y recursos. El mecanismo y el grano de la aceleración se precisarán a partir del estado del arte.

II. TRABAJO PREVIO Y ANTECEDENTES

El estado del arte ha abordado la aceleración de DFS y de recorridos de grafos desde tres frentes complementarios: la paralelización del recorrido, la optimización del acceso a memoria y la especialización en hardware. A continuación se revisan según la técnica que emplean para mejorar el desempeño.

II-A. Paralelización del recorrido

La estrategia más común consiste en descomponer el espacio de búsqueda en tareas independientes ejecutadas en paralelo, con balanceo dinámico de carga. Futuhi y Sturtevant proponen un marco CPU–GPU para DFS acotado por costo que desacopla las fases de búsqueda y de evaluación heurística, agrupa las evaluaciones en lotes y reparte los subárboles entre trabajadores asíncronos; el resultado es una aceleración de un orden de magnitud sobre problemas como el cubo de Rubik y los *sliding-tile puzzles*, sin perder la optimalidad del recorrido [4]. Su aporte para este trabajo es la evidencia de que las fases de un DFS pueden separarse y distribuirse, idea que reutilizamos al delegar el patrón de cómputo a hardware. Naumov *et al.* atacan el problema desde la estructura del grafo: presentan un algoritmo paralelo *work-efficient* que recorre un grafo acíclico dirigido en un estilo similar a BFS, a lo sumo tres veces, para obtener las relaciones de pre-order, post-order y padre de cada nodo; su implementación en CUDA alcanza hasta $6\times$ de aceleración frente al DFS secuencial en CPU [5]. Este trabajo aporta una técnica concreta para extraer paralelismo de un recorrido que la teoría clasifica como inherentemente secuencial. Yuan *et al.* llevan la misma idea al *subgraph matching*: organizan la exploración en una cola de tareas sin bloqueo (*lock-free*) repartida entre *warps* de la GPU, introducen un mecanismo de *timeout* que reasigna las tareas rezagadas para mantener el balance de carga, y emplean asignación dinámica de memoria para los espacios de pila [6]. De aquí tomamos el manejo del desbalance de carga entre ramas de profundidad desigual, un problema central en DFS. En conjunto, estos trabajos confirman que DFS expone paralelismo a nivel de tareas, pero todos dependen de plataformas CPU+GPU y de espacios de estados distintos a las matrices 2D densas que aborda esta propuesta.

II-B. Optimización del acceso a memoria y la localidad

Un segundo frente ataca el principal limitante del recorrido: los accesos irregulares a memoria. Dann *et al.* realizan una caracterización detallada de los patrones de acceso de varios aceleradores de grafos sobre FPGA y muestran que el rendimiento depende tanto de la capacidad de cómputo como de la organización de los datos en memoria; a partir de ello señalan que técnicas de partición de datos, procesamiento de regiones independientes y mejora de la localidad espacial y temporal reducen de forma notable el tiempo de acceso [3]. Su aporte es un catálogo de los factores de memoria que un acelerador de DFS debe atender. Mukkara *et al.* convierten ese diagnóstico en mecanismo: introducen **BDFS** (*Bounded Depth-First Scheduling*), una estrategia de planificación en línea que restringe cada núcleo a explorar una región conectada del grafo a la vez, y **HATS**, un planificador de recorrido acelerado por hardware que ejecuta dicha estrategia con un *overhead* de apenas 0,4% de área adicional sobre el núcleo base [7]. Este trabajo adopta una filosofía análoga a la de la presente propuesta: en lugar de reordenar vértices de un grafo general, se especializa el *datapath* de un núcleo RISC-V para ejecutar el patrón dominante del DFS sobre matrices

2D, trasladando el principio de aceleración ligada al patrón de recorrido a un dominio estructurado y de menor escala. SeqDFS persigue el mismo objetivo por software, reordenando el orden de visita de los vértices para mejorar la eficiencia de caché y reducir el costo del recorrido sobre grafos dispersos [8]; ambos coinciden en que el orden de visita es una palanca de optimización, idea aplicable al recorrido de una matriz. Giorgi aborda la limitación de memoria interna del dispositivo: como esta obliga a transferir datos de forma repetida —haciendo que el tiempo de transferencia domine sobre el de cómputo—, propone un particionamiento del grafo y una arquitectura distribuida multi-FPGA que supera a implementaciones equivalentes en CPU y GPU [2]. Estas soluciones evidencian que el manejo de memoria es tan decisivo como el cómputo, aunque se orientan a algoritmos iterativos sobre grafos de gran escala (como *PageRank*) y no al recorrido DFS sobre matrices.

II-C. Especialización en hardware reconfigurable

Un tercer frente mapea el recorrido o su lógica de control a *hardware* dedicado. Mukherji *et al.* implementan un DFS sobre FPGA como parte de un acelerador de codificación Huffman, demostrando que el recorrido puede mapearse de forma eficiente a hardware reconfigurable [9]; Mahammad *et al.* refuerzan esta vía con un codificador Huffman dual de selección dinámica de árboles en FPGA que reduce la latencia y el uso de recursos (LUTs) frente a trabajos previos, alcanzando alto *throughput* en aplicaciones de procesamiento de imágenes [10]. Ambos validan la pertinencia del hardware especializado para sistemas embebidos donde la velocidad y el uso de recursos son críticos. En el dominio del *backtracking*, Davis *et al.* descargan el lazo de propagación de restricciones de un solucionador SAT a un motor de inferencia en FPGA que almacena la instancia en Block RAM y particiona el problema entre varios motores en paralelo; el acelerador infiere implicaciones en 6 a 17 ciclos de reloj y resulta de 5 a 16 veces más rápido que la solución equivalente en software [11]. Su aporte es la prueba de que el lazo de avance y retroceso —el corazón del *backtracking*— puede ejecutarse en un *datapath* dedicado con ganancias sustanciales. No obstante, estos diseños atienden dominios cuyas estructuras de datos y lógica de poda difieren del recorrido sobre matrices 2D.

II-D. Fortalezas y debilidades de las soluciones del SOTA

La Tabla I resume, por frente de ataque, las fortalezas y debilidades identificadas en la Sección II, así como las condiciones bajo las cuales cada frente resulta viable o no para el problema de DFS sobre matrices 2D densas que aborda este trabajo. Se incorporan además tres frentes adicionales no cubiertos en la revisión original: las extensiones de ISA RISC-V orientadas a grafos (Yenimol *et al.* [12] y GPGCN [13]), los aceleradores de etiquetado de componentes conexas (CCL/CCA) sobre FPGA (Union-Retire [14], CCL 4K [15] y Run-Length CCA [16]), y los aceleradores de recorrido de grafos dispersos sobre FPGA con memoria de alto ancho de banda (ScalaBFS2 [17]), por compartir con

el recorrido DFS el patrón subyacente de exploración de vecindad sobre una estructura matricial o de grafo.

II-E. Desafíos observados

A partir del análisis de la Sección II y la síntesis presentada en la Tabla I, se identifican los principales desafíos que motivan este trabajo. En primer lugar, el paralelismo de DFS se ve limitado por la contención sobre el estado compartido del recorrido. En segundo lugar, el recorrido sobre matrices 2D presenta patrones de acceso a memoria irregulares, lo que reduce el aprovechamiento de caché y aumenta la latencia en arquitecturas de propósito general. En tercer lugar, las soluciones del estado del arte se enfocan principalmente en grafos dispersos, aceleración en GPU o dominios especializados como SAT o *subgraph matching*. Esto limita su aplicabilidad directa al problema de DFS sobre matrices 2D densas en un entorno embebido basado en RISC-V. En conjunto, estas limitaciones evidencian la ausencia de una arquitectura especializada que desacople el control del DFS del cómputo de exploración de vecinos, lo cual define el vacío arquitectónico que este trabajo aborda en la siguiente sección.

III. PROPUESTA DE SOLUCIÓN

III-A. Flujo de operación del acelerador DFS

La arquitectura propuesta implementa un acelerador hardware especializado para la ejecución del algoritmo Depth First Search (DFS) sobre matrices 2D. El sistema desplaza la carga computacional desde el procesador principal hacia una unidad dedicada en FPGA, compuesta por módulos de control, exploración y almacenamiento del estado de búsqueda. El flujo de operación se divide en las siguientes etapas:

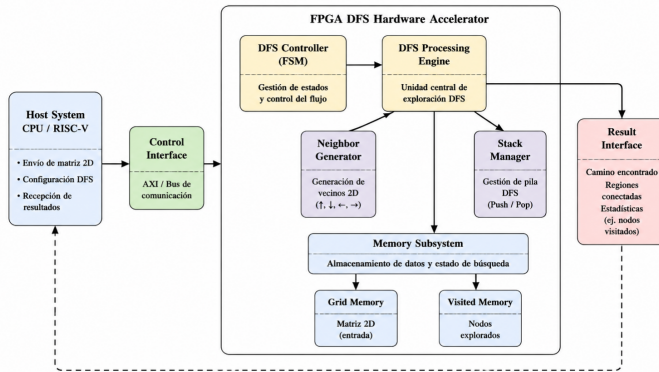


Figura 1. Arquitectura propuesta del acelerador hardware DFS para exploración de matrices 2D. El sistema separa la ejecución del algoritmo DFS del procesador principal mediante un motor especializado en FPGA compuesto por módulos de control, exploración y memoria.

III-A1. Configuración y carga de datos: Inicialmente, el sistema host, basado en un procesador CPU/RISC-V, envía la información necesaria para iniciar la búsqueda. Esta información incluye la matriz 2D de entrada, la posición inicial del recorrido y los parámetros de configuración del algoritmo. La matriz es almacenada en la memoria *Grid Memory*, mientras

que la memoria de nodos visitados (*Visited Memory*) es inicializada para evitar recorridos repetidos.

III-A2. Control de ejecución: El módulo *DFS Controller* administra la secuencia de operaciones del acelerador mediante una máquina de estados finitos (FSM). Este controlador coordina las etapas de lectura de nodos, exploración de vecinos, actualización del estado de memoria y finalización del recorrido. De esta forma, se mantiene el flujo de ejecución del algoritmo DFS sin requerir intervención continua del procesador principal.

III-A3. Generación de vecinos: El módulo *Neighbor Generator* se encarga de obtener las posiciones adyacentes al nodo actual dentro de la matriz 2D. Para una celda ubicada en la posición (x, y) , el hardware genera las posibles posiciones vecinas: $(x-1, y)$, $(x+1, y)$, $(x, y-1)$ y $(x, y+1)$. Esta operación permite realizar la exploración de la estructura matricial directamente en hardware, reduciendo el costo computacional asociado al cálculo repetitivo de vecinos.

III-A4. Administración de la pila DFS: Debido a la naturaleza iterativa del algoritmo DFS, es necesario mantener un registro de los nodos pendientes de exploración. El módulo *Stack Manager* implementa esta función mediante operaciones de almacenamiento y recuperación (*push/pop*), permitiendo conservar el orden de recorrido y administrar eficientemente el estado interno de la búsqueda.

III-A5. Actualización de memoria: Durante la exploración, el acelerador realiza accesos a las memorias internas para consultar la información de la matriz y actualizar los nodos visitados. La separación entre *Grid Memory* y *Visited Memory* permite mantener independiente la información de entrada y el estado dinámico del algoritmo, evitando exploraciones redundantes.

III-A6. Entrega de resultados: Finalmente, cuando no existen más nodos pendientes en la pila, el controlador finaliza la ejecución y el módulo *Result Interface* entrega la información generada al sistema host. Dependiendo de la aplicación, el resultado puede representar un camino encontrado, una región conectada o la cantidad de nodos explorados.

IV. RESULTADOS Y ANÁLISIS

IV-A. Plan de Experimento

Para contrastar la solución propuesta frente a una ejecución RISC-V base, se seleccionan cinco problemas de DFS sobre matrices 2D que cubren los patrones de cómputo identificados en la Sección II, tomados de LeetCode [20]. Cada uno se ejecuta variado para forzar su peor caso, midiendo el costo en ciclos simulados a distintos tamaños de matriz.

- **Number of Islands:** cuenta regiones de celdas conectadas en una matriz binaria, mediante flood fill que marca cada celda visitada y expande hacia sus vecinas. Se varía la conectividad (4/8) y la densidad de islas para medir el costo del patrón lineal en su mejor y peor caso.
- **Unique Paths III:** cuenta los caminos que visitan exactamente una vez cada celda transitable, mediante backtracking con marca y desmarca de visitado (*undo*). Se varía la densidad de obstáculos y el tamaño de la

matriz para medir el costo hasta el punto donde se vuelve impracticable.

- **Word Search II:** determina cuáles palabras de una lista pueden formarse recorriendo celdas adyacentes sin repetirlas, mediante backtracking guiado por un trie compartido que poda ramas sin prefijo válido. Se varía el tamaño del diccionario y se quita la poda para medir el costo con y sin esa optimización algorítmica.
- **Longest Increasing Path:** encuentra la longitud del camino más largo con valores estrictamente crecientes, mediante DFS multi-origen con memoización *top-down* para evitar recomputar subcaminos. Se corre con y sin memoización para medir el costo de la búsqueda exponencial frente a su versión acotada.
- **Pacific Atlantic Water Flow:** encuentra las celdas desde las que el agua puede fluir hacia ambos océanos en los bordes opuestos de la matriz, mediante DFS multi-origen lanzado desde los bordes con dos conjuntos de visitados que se intersectan al final. Se varía la forma de la matriz (cuadrada/angosta) para medir el costo de esa búsqueda anclada en el borde.

El acelerador propuesto se puede habilitar o deshabilitar en tiempo de ejecución, por lo que cada caso se corre dos veces sobre el mismo modelo –con el acelerador activo y con el núcleo RISC-V solo– y en ambas corridas se registra:

- Ciclos simulados (latencia)
- Instrucciones ejecutadas
- *Throughput* (celdas procesadas por ciclo)
- Celdas visitadas / nodos expandidos
- Profundidad máxima de pila alcanzada

Además, a partir de la síntesis e implementación del acelerador en la FPGA se reportan las métricas a nivel de *hardware*:

- Utilización de recursos (LUT, FF, BRAM y DSP)
- Frecuencia máxima de operación (*Fmax*) y cierre de *timing*
- *Throughput* sostenido en operaciones por segundo (OP/s)

El contraste entre ambas corridas, por métrica, cuantifica la ganancia del acelerador. Las herramientas son SystemC 2.3.4 con TLM 2.0 para el modelo y riscv64-unknown-elf-gcc/g++ para compilar el software; la implementación en *hardware* se realiza mediante síntesis de alto nivel (HLS) con AMD Vitis HLS [versión] y Vivado [versión], sobre una FPGA AMD Kria KV260.

DECLARACIÓN SOBRE EL USO DE INTELIGENCIA ARTIFICIAL

Durante la elaboración de este trabajo se utilizaron herramientas de inteligencia artificial generativa como apoyo en tareas de validación de información, mejora de la redacción y consolidación de contenido. En particular, se emplearon para contrastar afirmaciones técnicas contra las fuentes citadas, refinar la claridad y el estilo del texto, y consolidar la información de los trabajos del estado del arte. Un ejemplo del tipo de instrucción utilizada es: “consolida la información que te dimos de los 10 artículos y cítalos correctamente según la

técnica con que mejoran el DFS”. Todo el contenido técnico, las decisiones de diseño y la selección final de referencias fueron revisados y validados por los autores.

REFERENCIAS

- [1] DataCamp. (2024) Depth-first search (DFS) algorithm in Python. Tutorial en línea; consultado en 2026. [Online]. Available: <https://www.datacamp.com/es/tutorial/depth-first-search-in-python>
- [2] R. Giorgi, “Distributed large-scale graph processing on FPGAs,” *Journal of Big Data*, vol. 10, no. 1, p. 95, 2023.
- [3] J. Dann, D. Ritter, and H. Fröning, “Demystifying memory access patterns of FPGA-based graph processing accelerators,” in *Proc. 4th ACM SIGMOD Joint Int. Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, 2021.
- [4] E. Futuhi and N. R. Sturtevant, “A parallel CPU-GPU framework for cost-bounded DFS with applications to IDA* and BTS,” arXiv preprint arXiv:2507.11916, 2025, código basado en HOG2.
- [5] M. Naumov, A. Vrielink, and M. Garland, “Parallel depth-first search for directed acyclic graphs,” in *Proc. 7th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. ACM, 2017, pp. 1–8.
- [6] L. Yuan, D. Yan, J. Han, A. Ahmad, Y. Zhou, and Z. Jiang, “Faster depth-first subgraph matching on GPUs,” in *2024 IEEE 40th Int. Conf. on Data Engineering (ICDE)*, 2024.
- [7] A. Mukkara, N. Beckmann, M. Abeydeera, X. Ma, and D. Sánchez, “Exploiting locality in graph analytics through hardware-accelerated traversal scheduling,” in *2018 51st Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2018.
- [8] Q. Lyu, M. Sha, B. Gong, and K. Lyu, “Accelerating depth-first traversal by graph ordering,” in *Proc. Int. Conf. on Scientific and Statistical Database Management (SSDBM)*, 2021.
- [9] P. Mukherji, S. Rajput, and P. Mankar, “Depth-first search based accelerated Huffman encoding on FPGA,” in *2025 IEEE 34th Int. Conf. on Microelectronics (MIEL)*, 2025.
- [10] M. Mahammad, A. Uppala, S. Hussain, and A. Marouthu, “FPGA implementation of high-performance Huffman encoder for image processing applications,” *International Journal of Reconfigurable and Embedded Systems (IJRES)*, vol. 15, no. 1, pp. 68–77, 2026.
- [11] J. Davis, Z. Tan, F. Yu, and L. Zhang, “Designing an efficient hardware implication accelerator for SAT solving,” in *Int. Conf. on Theory and Applications of Satisfiability Testing (SAT)*, ser. Lecture Notes in Computer Science, vol. 4996. Springer, May 2008, pp. 48–62.
- [12] M. S. Yenimol, G. Pulat, and O. Ozturk, “RISC-V instruction set extension for graph applications,” in *Proceedings of the Sixth Workshop on Computer Architecture Research with RISC-V (CARRV'22)*. New York, NY, USA: ACM, 2022, pp. 1–5.
- [13] W. Tang and P. Zhang, “GPGCN: A general-purpose graph convolution neural network accelerator based on RISC-V ISA extension,” *Electronics*, vol. 11, no. 22, p. 3833, 2022.
- [14] D. G. Bailey and M. J. Klaiber, “Union-retain for connected components analysis on FPGA,” *Journal of Imaging*, vol. 8, no. 4, p. 89, 2022.
- [15] M. Kowalczyk, P. Ciarach, D. Przewlocka-Rus, H. Szolc, and T. Kryjak, “Real-time FPGA implementation of parallel connected component labelling for a 4K video stream,” *Journal of Signal Processing Systems*, vol. 93, pp. 481–498, 2021.
- [16] K. Appiah, A. Hunter, P. Dickinson, and J. D. Owens, “A run-length based connected component algorithm for FPGA implementation,” in *International Conference on Field-Programmable Technology (FPT)*, Taipei, Taiwan, Dec. 2008, pp. 177–184.
- [17] K. Li, S. Xu, Z. Shao, R. Zheng, X. Liao, and H. Jin, “ScalaBFS2: A high-performance BFS accelerator on an HBM-enhanced FPGA chip,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 2, pp. 1–39, 2024.
- [18] Y. Ham *et al.*, “Graphicionado: A high-performance and energy-efficient accelerator for graph analytics,” in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.
- [19] G. Dai *et al.*, “Foregraph: Exploring large-scale graph processing on multi-fpga architecture,” in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2017.
- [20] LeetCode. (2026) LeetCode – algorithm problem set. Plataforma de práctica de algoritmos; consultado en 2026. [Online]. Available: <https://leetcode.com>

Tabla I
FORTALEZAS, DEBILIDADES Y VIABILIDAD DE LAS SOLUCIONES DEL SOTA, POR FRENTE DE ATAQUE.

Frente	Refs.	Fortalezas	Debilidades / no viable cuando...	Viable cuando...
Paralelización del recorrido (CPU+GPU)	[4], [5], [6]	Extraen paralelismo de tareas de un algoritmo formalmente secuencial; ganancias de un orden de magnitud (DFS acotado) hasta $6\times$ (DAG en CUDA); resuelven desbalance de carga entre ramas de profundidad desigual.	Dependen de la dupla CPU+GPU como plataforma, no de un núcleo embebido aislado; el espacio de estados es un grafo general o un puzzle, no una matriz 2D con vecindad fija de 4 direcciones.	Se dispone de GPU y el dominio es un grafo/DAG general o un espacio de estados tipo puzzle con heurística costosa de evaluar.
Optimización de memoria y localidad	[2], [3]	Catalogan los factores de memoria que dominan el costo del recorrido (partición, localidad espacial/temporal, transferencia repetida); la arquitectura multi-FPGA de Giorgi supera a CPU/GPU equivalentes cuando el grafo no cabe en un solo dispositivo.	Dann et al. es un estudio de caracterización, no propone una arquitectura ejecutable; Giorgi paga el costo de transferencia entre FPGAs si el grafo excede la memoria interna de un dispositivo; ambos atienden algoritmos iterativos (p. ej. PageRank) sobre grafos dispersos de gran escala, no DFS sobre matrices densas.	El grafo es de gran escala, no cabe en la memoria interna de un solo FPGA, y el algoritmo es iterativo tipo PageRank.
Especialización en hardware reconfigurable (Huffman/SAT)	[9], [10], [11]	Prueban que el lazo de avance/retroceso del backtracking puede ejecutarse en un datapath dedicado con ganancias sustanciales ($5\text{--}16\times$ sobre software; $6\text{--}17$ ciclos por implicación en SAT); bajo uso de recursos (LUTs) y alto throughput en codificación Huffman.	El recorrido DFS está embebido y acoplado a la lógica del dominio (codificación Huffman o propagación de cláusulas SAT); no es extraíble como motor de recorrido genérico reutilizable para otro problema.	La aplicación es compresión (Huffman) o SAT solving, donde la estructura de datos y la lógica de poda del dominio ya están fijadas de antemano.
<i>Frentes adicionales identificados (no cubiertos en la revisión original):</i>				
Extensión de ISA RISC-V para grafos	[12], [13]	Ofrecen programabilidad vía instrucciones personalizadas sin sacrificar aceleración; GPGCN reporta más de $1001\times$ y $267\times$ de aceleración frente a CPU tradicional y CPU con extensión vectorial, respectivamente, sobre el dataset Cora.	Yenimol et al. presenta solo el diseño de la micro-arquitectura y un estudio inicial (70 % de reducción en accesos bloqueantes), <i>sin implementación física ni resultados reproducibles en hardware</i> ; GPGCN está acoplado al cómputo de GCN sobre matrices dispersas en formato CSR, no al patrón de pila (<i>push/pop</i>) de un DFS.	Como precedente metodológico de que una ISA RISC-V extendida puede acelerar cómputo irregular sobre estructuras tipo grafo; no como solución lista para contrastar cuantitativamente con DFS sobre matrices.
Etiquetado de componentes conexas (CCL/CCA) sobre FPGA	[14], [15], [16]	Resuelven en hardware un caso particular de recorrido tipo <i>flood fill</i> sobre matrices binarias con muy bajo uso de recursos: Union-Retire ahorra 36 % de memoria frente al barrido zig-zag; el módulo de Kowalczyk et al. sostiene 60 fps en video 4K/UHD; Run-Length logra hasta $15\times$ sobre una implementación equivalente en software.	Todos están especializados en el patrón <i>single-pass/raster-scan</i> sobre imágenes binarias (conectividad de píxeles), no en el recorrido DFS general con pila explícita, múltiples orígenes o <i>backtracking</i> con deshacer (<i>undo</i>) que exige este proyecto; el ahorro de memoria de Union-Retire se paga con +17 % de lógica y +36 % de registros; el módulo de Kowalczyk et al. degrada si ocurre más de un <i>merger</i> sostenido por ciclo.	La matriz es binaria y basta una sola pasada <i>raster</i> (sin necesidad de pila explícita ni <i>backtracking</i>).
Aceleradores de grafos dispersos en FPGA con HBM	[17]	Escala el cómputo de BFS según los canales de memoria HBM disponibles mediante un <i>crossbar</i> multi-capas y PEs de modo híbrido; ScalaBFS2 alcanza 56.92 GTEPS operando a 170–225 MHz sobre el chip Xilinx XCU280, casi $3\times$ el rendimiento de su versión anterior (ScalaBFS, 19.7 GTEPS) y equivalente al de GPUs de gama alta con la mitad del ancho de banda.	Requiere una FPGA con HBM (plataforma específica y costosa, p. ej. Alveo/XCU280, distinta a la KV260 disponible en este curso); está orientado a BFS sobre grafos dispersos de gran escala en formato vértice-céntrico, no a DFS con pila sobre matrices densas de tamaño pequeño/mediano; el costo del <i>crossbar</i> de despacho de vértices crece con el cuadrado del número de PEs, por lo que una matriz completa puede consumir más de la mitad de los LUTs disponibles si el número de PEs es grande, limitando cuánto puede escalar el diseño antes de agotar la lógica.	Se dispone de una FPGA con HBM y el cuello de botella del problema es el ancho de banda de acceso aleatorio sobre un grafo disperso de gran escala procesado vértice por vértice.
Un acelerador de alto rendimiento y eficiencia energética para análisis de grafos	[18]	Presenta una arquitectura especializada para algoritmos de grafos que integra procesamiento y memoria cercana al cómputo. Reduce significativamente los accesos a memoria externa y mejora el rendimiento energético frente a CPU y GPU.	Está orientado a grafos generales de gran escala almacenados en formatos dispersos. La complejidad del hardware y los requerimientos de memoria pueden resultar excesivos para problemas basados en matrices 2D de tamaño moderado.	Es especialmente adecuado cuando se procesan grafos muy grandes donde el cuello de botella principal es el acceso a memoria y se requiere maximizar throughput y eficiencia energética.
Procesamiento de gráficos a gran escala en arquitecturas multi-FPGA	[19]	Demuestra cómo particionar grafos y distribuir el procesamiento entre múltiples FPGA para escalar el rendimiento. Proporciona estrategias efectivas para balanceo de carga y comunicación entre dispositivos.	La complejidad de implementación aumenta considerablemente debido a la necesidad de múltiples FPGA y mecanismos de sincronización. No resulta práctico para sistemas embebidos o plataformas de bajo costo.	Es viable cuando se trabaja con grafos extremadamente grandes que exceden la capacidad de una única FPGA y se dispone de infraestructura multi-dispositivo.